

Projektowanie i Analiza Algorytmów	
Kierunek <i>Informatyczne Systemy Automatyki</i>	Termin <i>wtorek 17⁰⁵ – 18⁴⁵</i>
Imię, nazwisko, numer albumu <i>Jan Hulbój 284197</i>	Data <i>14.04.2026</i>
Link do projektu https://github.com/mlody-jano/algorithm_design_and_analysis	



Politechnika
Wrocławska

SPRAWOZDANIE – MINIPROJEKT NR 1

Poniższy dokument jest kompleksowym sprawozdaniem opisującym badania przeprowadzone w ramach pierwszego projektu na przedmiocie **Projektowanie i Analiza Algorytmów**. Przedmiotem pierwszego projektu były **algorytmy sortowania**, gdzie zaimplementowane zostały 3 wybrane algorytmy, a następnie zbadano ich złożoność obliczeniową. Algorytmy, które zostały zaimplementowane to: **mergeSort**, **quickSort** oraz **introSort**. Dane uzyskane podczas prowadzenia testów, zostały zaprezentowane w postaci wykresów oraz tabel. Testowy zbiór danych dla którego zostały prowadzone badania jest okrojoną bazą filmów z www.kaggle.com zatytułowaną *"IMDB Largest Review Dataset"*.

Spis treści

1 Wstęp teoretyczny	2
1.1 Sortowanie przez scalanie	2
1.2 Sortowanie szybkie	2
1.3 Sortowanie introspektywne	2
2 Założenia projektowe	3
3 Wyniki badań	3
3.1 Wczytanie i filtracja danych	3
3.2 Sortowanie przez scalanie	4
3.3 Sortowanie szybkie	5
3.4 Sortowanie introspektywne	5
3.5 Porównanie złożoności wszystkich algorytmów	6
4 Wnioski	6

Spis rysunków

1	Fragment kodu odpowiedzialny za pomiar czasu algorytmów sortowania.	3
2	Złożoność obliczeniowa ładowania danych do struktury.	4
3	Złożoność obliczeniowa algorytmu mergeSort()	5
4	Złożoność obliczeniowa algorytmu quickSort()	5
5	Złożoność obliczeniowa algorytmu introSort()	6
6	Złożoności obliczeniowe algorytmów sortujących	6

Spis tabel

1	Złożoności obliczeniowe mergeSort()	2
2	Złożoności obliczeniowe quickSort()	2

3	Złożoności obliczeniowe introSort()	3
4	Wyniki badań czasu ładowania danych	4
5	Wyniki badań czasu algorytmu mergeSort()	4
6	Wyniki badań czasu algorytmu quickSort()	5
7	Wyniki badań czasu algorytmu introSort()	6

1 Wstęp teoretyczny

1.1 Sortowanie przez scalanie

Sortowanie przez scalanie (*mergeSort*) jest algorytmem sortowania bazującym na zasadzie "dziel i rządź", ponieważ dzieli on listę na mniejsze podlisty, aż do uzyskania listy pojedynczych. Następnie porównuje kolejne elementy i scala je w odpowiedniej kolejności w coraz większe listy, końcowo tworząc posortowaną listę oryginalnej długości. Proces sortowania w tym algorytmie zachodzi dopiero pod sam koniec, tj. po podziale na pojedyncze listy, co może skutkować dłuższym czasem wykonania.

Dodatkowo, zastosowano jednokrotnie alokowany bufor zmniejszający ilość rekursywnych alokacji pamięci, co redukuje wartość złożoności obliczeniowej. Bufor ten jest definiowany w publicznie wywoływanej funkcji **mergeSort()** i jest przekazywany jako wskaźnik przez argument do prywatnych funkcji wykonujących faktyczne sortowanie.

Oczekiwane złożoności obliczeniowe **sortowania przez scalanie** wynoszą:

Tabela 1: Złożoności obliczeniowe **mergeSort()**

Lp.	Nazwa	Optymistyczna	Średnia	Pesymistyczna
1.	mergeSort()	$O(n\log(n))$	$O(n\log(n))$	$O(n\log(n))$

1.2 Sortowanie szybkie

Sortowanie szybkie (*quickSort*) opiera się na wyborze elementu rozdzielającego tj. **pivota**, który dzieli tablicę. Następnie wartości mniejsze od elementu rozdzielającego są przenoszone na lewą stronę listy, a elementy większe na prawą stronę. Algorytm działa rekurencyjnie do momentu uzyskania tablicy jednoelementowej, niewymagającej posortowania. Wrażliwym punktem algorytmu sortowania jest jego pesymistyczna złożoność obliczeniowa $O(n^2)$, która może wystąpić w przypadku gdy nieodpowiednio wybrany zostanie element rozdzielający, lub zbiór danych jest już częściowo posortowany.

Oczekiwane złożoności obliczeniowe **sortowania szybkiego** wynoszą:

Tabela 2: Złożoności obliczeniowe **quickSort()**

Lp.	Nazwa	Optymistyczna	Średnia	Pesymistyczna
1.	quickSort()	$O(n\log(n))$	$O(n\log(n))$	$O(n^2)$

Ze względu na znaczną obecność rekordów o takich samych wartościach rankingu w sortowanej bazie danych, zastosowany został zmodyfikowany wariant algorytmu sortowania szybkiego, zakładający podział listy na trzy regiony: $x < pivot$, $x > pivot$ oraz $x = pivot$. Taka modyfikacja pozwala na znaczącą redukcję rekursywnych wywołań sortujących elementy o tych samych wartościach, które już są posortowane.

Dodatkowo, celem zmniejszenia prawdopodobieństwa wystąpienia najgorszego przypadku algorytmu, w którym wpada on w złożoność $O(n^2)$, zastosowane zostało wyznaczenie pivota na podstawie trzech elementów zbioru: na pierwszej pozycji, na ostatniej pozycji oraz na środkowej pozycji. Nie jest to jednak całkowite wyeliminowanie możliwości wystąpienia przypadku pesymistycznego, co w praktyce rozwiązuje algorytm **introSort()**.

1.3 Sortowanie introspektywne

Sortowanie introspektywne (*introSort*) jest hybrydowym algorytmem łączącym algorytmy **quickSort()**, **heapSort()** oraz **insertionSort()**. Algorytm początkowo wykonuje **quickSort()** na zbiorze danych, natomiast w sytuacji przekroczenia wybranego progu głębokości rekurencji algorytm sortujący zmieniany jest na **heapSort()**, który zapobiega wpadnięciu w złożoność obliczeniową $O(n^2)$. Dodatkowo, dla

zbiorów danych mniejszych od określonego progu, algorytm sortowania przełącza oryginalne sortowanie `quickSort()` na sortowanie `insertionSort()`.

Wartość maksymalnej głębokości rekurencji, po przekroczeniu której algorytm `quickSort()` jest przełączany, została określona na podstawie następującego wzoru:

$$depthLimit = 2 \cdot \log(table.returnSize()) \quad (1)$$

, gdzie *size* oznacza ilość elementów obecnie zawartych w strukturze.

Oczekiwane złożoności obliczeniowe **sortowania introspektywnego** wynoszą:

Tabela 3: Złożoności obliczeniowe **introSort()**

Lp.	Nazwa	Optymistyczna	Średnia	Pesymistyczna
1.	<code>quickSort()</code>	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$

2 Założenia projektowe

W celu zapewnienia odpowiedniego podłoża merytorycznego do wyciągnięcia wniosków dotyczących badanych algorytmów, przyjęte zostały następujące założenia:

- **Rozmiary struktur:** 10000, 100000, 500000, 100000, maksymalna ilość rekordów z pliku
- **Ilość powtórzeń operacji:** 10
- **Sposób pomiaru czasu:** Pomiar czasu z wykorzystaniem `high_resolution_clock`.

Testy utworzonych struktur danych były prowadzone na urządzeniu wyposażonym w następujące komponenty:

- **Procesor** - AMD Ryzen 7 2700X 3,7Ghz
- **RAM** - 16GB

Badanie czasu operacji zostało zrealizowane w postaci wykorzystania biblioteki `chrono`, która zawiera implementację klasy `high_resolution_clock`, która swoją funkcją `now()`, pozwala na dokładny pomiar czasu pracy algorytmów.

```
if(algo == "quick") {
    auto start = high_resolution_clock::now();
    table.quickSort(0, table.returnSize() - 1);
    auto end = high_resolution_clock::now();
    sortTime = duration_cast<nanoseconds>(end - start).count();
}
else if(algo == "merge") {
    auto start = high_resolution_clock::now();
    table.mergeSort(0, table.returnSize() - 1);
    auto end = high_resolution_clock::now();
    sortTime = duration_cast<nanoseconds>(end - start).count();
}
else if(algo == "intro") {
    int depthLimit = 2 * log(table.returnSize());
    auto start = high_resolution_clock::now();
    table.introSort(0, table.returnSize() - 1, depthLimit);
    auto end = high_resolution_clock::now();
    sortTime = duration_cast<nanoseconds>(end - start).count();
}
```

Rysunek 1: Fragment kodu odpowiedzialny za pomiar czasu algorytmów sortowania.

3 Wyniki badań

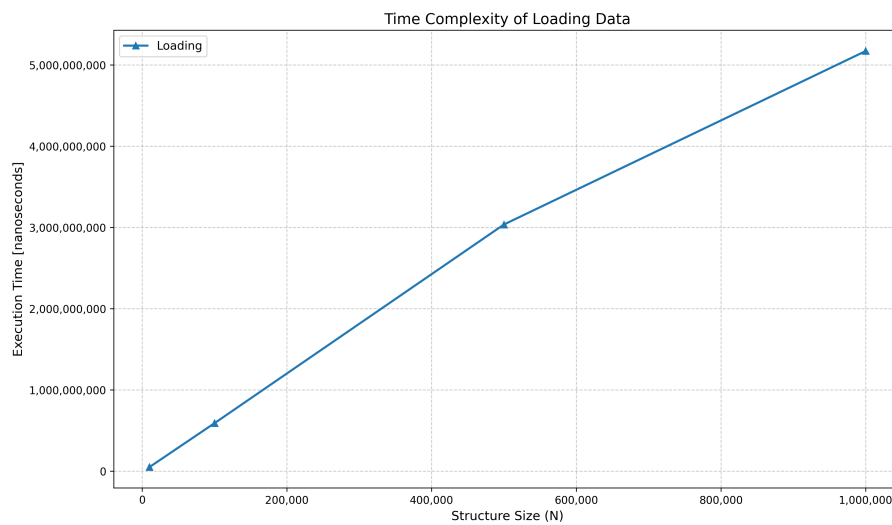
3.1 Wczytanie i filtracja danych

Przed rozpoczęciem badań nad złożonościami obliczeniowymi algorytmów, dane z zewnętrznego zbioru danych muszą zostać oczyszczone oraz odfiltrowane przed załadowaniem ich do struktury danych. W

głównej pętli programu zawarte zostały funkcje parsujące oraz czyszczące rekordy pobierane z bazy, oraz usuwające rekordy puste. Oczekiwana złożoność obliczeniowa procesu ładowania danych jest równa $O(n)$ za względu na n-krotne wykonanie operacji $O(1)$. Proces ładowania danych do struktury był badany pod kątem czasu, co zostało zaprezentowane w poniższej tabeli oraz na wykresie:

Tabela 4: Wyniki badań czasu **ładowania danych**.

Lp.	N	Czas(ms)
1.	10000	49,96
2.	100000	590,20
3.	500000	3029,10
4.	962903	5164,00



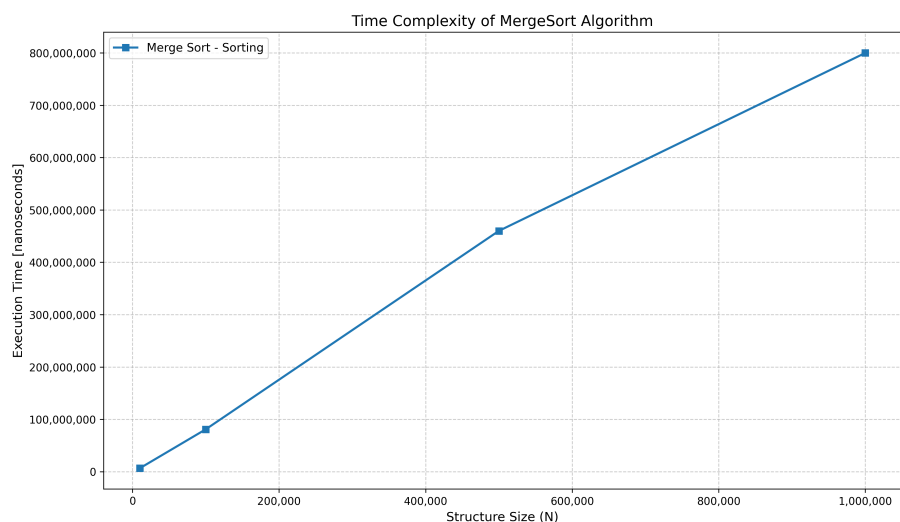
Rysunek 2: Złożoność obliczeniowa ładowania danych do struktury.

3.2 Sortowanie przez scalanie

Wyniki badań prowadzonych dla algorytmu **sortowania przez scalanie** zostały zaprezentowane na poniższej tabeli oraz wykresie:

Tabela 5: Wyniki badań czasu algorytmu **mergeSort()**.

Lp.	N	Czas(ms)	$\bar{x}$	M_d
1.	10000	6,547	5,43	5.0
2.	100000	80,933	6,13	7.0
3.	500000	459,940	6,59	7.0
4.	962903	799,797	6,58	7.0



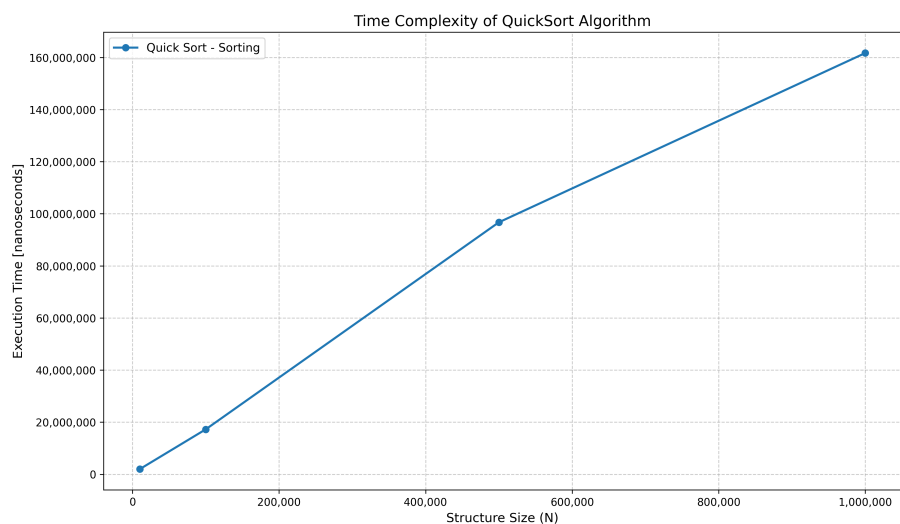
Rysunek 3: Złożoność obliczeniowa algorytmu **mergeSort()**.

3.3 Sortowanie szybkie

Wyniki badań prowadzonych dla algorytmu **sortowania szybkiego** zostały zaprezentowane na poniższej tabeli oraz wykresie:

Tabela 6: Wyniki badań czasu algorytmu **quickSort()**.

Lp.	N	Czas(ms)	$\bar{x}$	M_d
1.	10000	1,989	5,43	5.0
2.	100000	17,241	6,13	7.0
3.	500000	96,726	6,59	7.0
4.	962903	161,686	6,58	7.0



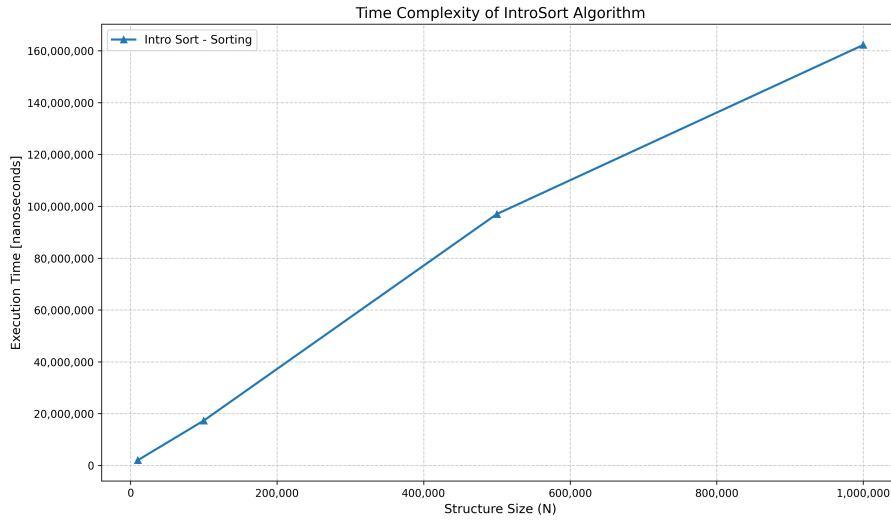
Rysunek 4: Złożoność obliczeniowa algorytmu **quickSort()**.

3.4 Sortowanie introspektywne

Wyniki badań prowadzonych dla algorytmu **sortowania introspektywnego** zostały zaprezentowane na poniższej tabeli oraz wykresie:

Tabela 7: Wyniki badań czasu algorytmu **introSort()**.

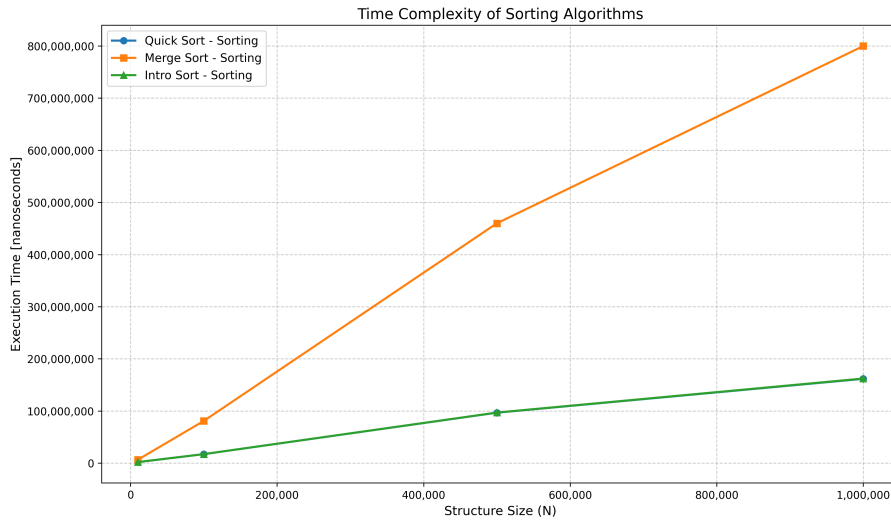
Lp.	N	Czas(ms)	$\bar{x}$	M_d
1.	10000	1,996	5,43	5.0
2.	100000	17,266	6,13	7.0
3.	500000	97,005	6,59	7.0
4.	962903	162,246	6,58	7.0



Rysunek 5: Złożoność obliczeniowa algorytmu **introSort()**.

3.5 Porównanie złożoności wszystkich algorytmów

Wyniki badań zestawione na jednym wykresie w celu lepszej wizualizacji różnic zostały przedstawione na poniższym wykresie:



Rysunek 6: Złożoności obliczeniowe algorytmów sortujących

4 Wnioski

Przeprowadzone badania, których wyniki zostały przedstawione w powyższym sprawozdaniu jasno pokazują różnice i podobieństwa każdego z algorytmów. Wykresy utworzone z danych uzyskanych podczas prowadzenia testów obrazują jak wyglądają złożoności obliczeniowe każdego z algorytmów.

Algorytm `mergeSort()` okazał się być najbardziej czasochłonnym algorytmem, posiadając większą wartość stałą niż pozostałe algorytmy, zachowując złożoność obliczeniową zgodną z oczekiwaniami, tj. $O(n \log(n))$. Większy czas sortowania, spowodowany jest między innymi przed wielokrotne rekurencyjne wywoływanie scalań list, wymagającego przepisywania elementów. Zastosowane usprawnienie z wykorzystaniem jednokrotnie alokowanego bufora zmniejszyło narzut czasowy, redukując liczbę alokacji pamięci, natomiast wciąż jest on znacznie większy niż w pozostałych algorytmach.

Drugim pod względem wyników złożoności czasowej jest algorytm `introSort()`, dla którego uzyskane zostały wyniki bardzo zbliżone do algorytmu `quickSort()`. Może to być spowodowane bardzo rzadkim przełączaniem algorytmu sortującego na inne, tj. `insertionSort()`, bądź `heapSort()`. Oznacza to, że algorytm `quickSort()`, którym początkowo operuje `introSort()`, sprawdza się dobrze i nie wpada w głębokie pętle rekurencyjne, powodujące wpadanie w złożoność $O(n^2)$.

Najlepszym algorytmem badanym okazał się w tym przypadku `quickSort()`, którego wyniki złożoności czasowej okazały się być bardzo zbliżone do wyników, algorytmu `introSort()`. Poprzez zastosowanie podziału listy na trzy regiony, algorytm nie traci czasu na rekurencyjne sprawdzanie kolejności elementów równych, co znacząco przyspiesza czas wykonania.

Bibliografia

1. GOODRICH, Michael T.; TAMASSIA, Roberto; MOUNT, David M. *Data Structures and Algorithms in C++*. 2nd ed. Hoboken, N.J: Wiley, 2011. ISBN 978-0-470-38327-8.
2. Introsort. W: *Wikipedia online*[b.d.] [dostęp: 2026-04-10]. Dostępne z: <https://en.wikipedia.org/w/index.php?title=Introsort&oldid=1318479533>.
3. Merge Sort. W: *Wikipedia online*[b.d.] [dostęp: 2026-04-10]. Dostępne z: https://en.wikipedia.org/w/index.php?title=Merge_sort&oldid=1346352707.
4. Quicksort. W: *Wikipedia online*[b.d.] [dostęp: 2026-04-10]. Dostępne z: <https://en.wikipedia.org/w/index.php?title=Quicksort&oldid=1347087395>.